

Bulk-loading de R-trees

CC4102 – Diseño y Análisis de Algoritmos
Tarea 1

Integrantes: Javier Andreu Diaz Parada
Profesor: Benjamín Bustos
Fecha de realización: 30-04-2026
Fecha de entrega: 10-05-2026

1. Introducción

Esta tarea estudia la construcción masiva de R-trees estáticos para puntos en dos dimensiones, considerando dos métodos de *bulk-loading*: Nearest-X y Sort-Tile-Recursive (STR). El árbol se construye primero en memoria principal, luego se serializa en un archivo binario con un nodo por bloque de 4096 bytes, y finalmente las consultas por rango se ejecutan leyendo nodos desde disco.

El informe presenta las decisiones principales de implementación, la metodología experimental usada para comparar ambos métodos, los resultados de construcción y consulta, y un análisis de las diferencias observadas. La comparación se centra en dos costos: el tiempo necesario para construir y escribir cada árbol, y la cantidad de lecturas de nodo requeridas durante consultas por rango rectangular.

La hipótesis inicial es que STR tendrá un mayor costo de construcción que Nearest-X, porque realiza ordenamientos y particiones adicionales; sin embargo, debería construir nodos con MBR más compactos espacialmente y, por lo tanto, reducir la cantidad de lecturas de nodos durante las consultas. En cambio, se espera que Nearest-X sea más rápido al construir, pero menos eficiente al consultar, ya que agrupa usando principalmente la coordenada X.

2. Desarrollo

2.1. Representación de nodos y serialización

La implementación usa un R-tree estático para puntos 2D. Cada nodo se representa con un campo `k`, un arreglo fijo de 204 entradas y un relleno de 12 bytes. Cada entrada contiene un MBR con cuatro float, en el orden (`x1`, `x2`, `y1`, `y2`), y un entero `child`. Cuando la entrada pertenece a una hoja, `child` = -1; cuando apunta a un nodo interno, `child` corresponde al índice del nodo hijo dentro del vector de nodos y, posteriormente, dentro del archivo serializado.

Con esta representación, cada entrada ocupa 20 bytes: 16 bytes del MBR y 4 bytes del índice del hijo. Por lo tanto, el tamaño del nodo queda dado por:

$$4 + 204 \cdot 20 + 12 = 4096 \text{ bytes}$$

Esta igualdad permite que cada nodo coincida con un bloque lógico de disco. En el código se agregaron verificaciones con `static_assert` para confirmar los tamaños esperados de las estructuras principales. La serialización se realiza escribiendo las estructuras completas al archivo binario. Esta decisión simplifica la implementación, pero asume compatibilidad de representación binaria, alineamiento y *endianness* con la máquina donde se ejecuta. Por lo tanto, durante las consultas cada lectura de nodo corresponde a una lectura de bloque lógico.

2.2. Construcción en memoria

Durante la construcción, el árbol se mantiene como un `std::vector<Node>`. La posición 0 se reserva para la raíz, siguiendo el formato pedido. Los demás nodos se agregan secuencialmente al vector. Al terminar la construcción, el vector se escribe completo en disco en el mismo orden, de modo que el índice de cada nodo coincide con su posición en el archivo.

Para leer el nodo `i` durante una consulta, se usa el desplazamiento:

$$i \cdot \text{sizeof}(\text{Node})$$

La función `readNode` encapsula esta lectura y aumenta un contador de I/O lógico cada vez que se lee un nodo completo desde el archivo binario. El contador reportado en los resultados corresponde a esta cantidad de lecturas de bloques/nodos.

2.3. Nearest-X

Nearest-X se implementó como un procedimiento *bottom-up*. Al comenzar, cada punto se transforma en una entrada hoja cuyo MBR tiene área cero. En cada nivel se ordenan las entradas por la coordenada X del centro de su MBR y luego se agrupan consecutivamente en bloques de hasta 204 entradas. Cada grupo forma un nodo del nivel actual.

Después de crear los nodos de un nivel, se calcula el MBR de cada nodo y se genera una entrada padre que apunta a la posición del nodo recién creado. Si todas las entradas padre caben en un solo nodo, se construye la raíz en la posición 0. Si no caben, el proceso se repite con las entradas padre.

La decisión relevante de implementación es que el mismo mecanismo se usa tanto para el nivel de hojas como para los niveles internos. Esto evita tener dos constructores separados y mantiene consistente el cálculo de MBR e índices. En la práctica, esta construcción resulta más barata que STR porque el ordenamiento de cada nivel considera solo una clave espacial principal.

2.4. Sort-Tile-Recursive

STR también se implementó de forma *bottom-up*, pero con un empaquetamiento espacial en dos coordenadas. Para un nivel con n entradas y capacidad $b = 204$, se calcula la cantidad aproximada de páginas:

$$P = \left\lceil \frac{n}{b} \right\rceil$$

Luego se define la cantidad de franjas verticales como:

$$S = \left\lceil \sqrt{P} \right\rceil$$

Las entradas se ordenan por la coordenada X del centro del MBR y se dividen en s franjas. Dentro de cada franja, las entradas se ordenan por la coordenada Y del centro del MBR y se agrupan en nodos de hasta 204 entradas. Al igual que en Nearest-X, luego se calculan los MBR de los nodos y se generan entradas padre para el siguiente nivel.

La diferencia principal con el pseudocódigo ideal es el uso de `ceil`, porque la cantidad de entradas no necesariamente produce un cuadrado perfecto de páginas. Con esto se evita perder entradas o generar divisiones incompletas mal definidas. El objetivo de STR es que los MBR resultantes sean más compactos en X e Y, aun pagando más tiempo de construcción.

2.5. Consultas por rango

Las consultas reciben un rectángulo $R = \{x_{min}, x_{max}, y_{min}, y_{max}\}$. La búsqueda comienza leyendo la raíz desde el archivo binario. Para cada entrada del nodo leído se verifica si su MBR intersecta el rectángulo de consulta. Si no intersecta, se poda esa rama. Si intersecta y `child != -1`, se lee recursivamente el nodo hijo. Si `child = -1`, la entrada representa un punto y se verifica si está contenido en el rectángulo.

Las comparaciones de intersección y contención son inclusivas en los bordes. Esta decisión evita excluir puntos ubicados exactamente sobre el límite de una consulta. El costo principal medido en consultas no es el tiempo de CPU, sino la cantidad de nodos leídos desde disco. Todas las consultas reportadas se ejecutaron sobre el archivo binario serializado, no sobre el vector usado durante la construcción.

3. Metodología experimental

Se usaron los dos datasets entregados por la tarea. El primero contiene puntos aleatorios uniformemente distribuidos en $[0, 1] \times [0, 1]$. El segundo contiene ubicaciones de edificios de Europa, normalizadas al mismo rango. Ambos datasets están almacenados como pares de `float` con coorde-

nadas X e Y. En total se realizaron 40 construcciones: 10 tamaños de entrada por 2 datasets por 2 métodos. Cada combinación se ejecutó una vez.

Para construcción se usaron los primeros N puntos de cada dataset, con:

$$N \in \{2^{15}, 2^{16}, \dots, 2^{24}\}$$

Para cada valor de N se construyeron cuatro árboles: random con Nearest-X, random con STR, Europa con Nearest-X y Europa con STR. En esta versión se ejecutó una construcción por combinación de dataset, método y tamaño. El tiempo reportado corresponde al tiempo total de construcción y escritura del árbol. En total se ejecutaron 2000 consultas: 2 datasets por 2 métodos por 5 valores de s por 100 consultas.

Para consultas se usaron solo los cuatro árboles contruidos con $N = 2^{24}$. Para cada lado de cuadrado:

$$s \in \{0.0025, 0.005, 0.01, 0.025, 0.05\}$$

se generaron 100 consultas cuadradas aleatorias dentro del rango $[0, 1] \times [0, 1]$. Se usó una semilla fija para hacer reproducibles las consultas. Para cada árbol y cada valor de s , se midió la cantidad promedio de lecturas de nodo, el tiempo promedio de consulta, el promedio de puntos reportados y la desviación estándar de los puntos reportados. El tiempo reportado corresponde al tiempo total de construcción más escritura del árbol a disco.

3.1. Ambiente de ejecución

Recurso	Valor
Equipo	HP Victus by HP Gaming Laptop 16-s1xxx
CPU	AMD Ryzen 7 8845HS w/ Radeon 780M Graphics
Cores / threads	8 / 16
Caché L2	8192 KB
Caché L3	16384 KB
RAM	16440479744 bytes, aproximadamente 15.31 GB
Sistema operativo	Microsoft Windows 11 Pro 64-bit, versión 10.0.26200
CMake	4.2.3-msvc3

4. Resultados

4.1. Construcción

La Tabla 1 resume los resultados para el mayor tamaño probado, $N = 2^{24}$.

Dataset	Método	Nodos	Altura	Tiempo total (ms)
random	Nearest-X	82649	4	21394.891
random	STR	82649	4	34226.894
europa	Nearest-X	82649	4	21149.141
europa	STR	82649	4	33126.152

Tabla 1: Resumen de construcción para $N = 2^{24}$.

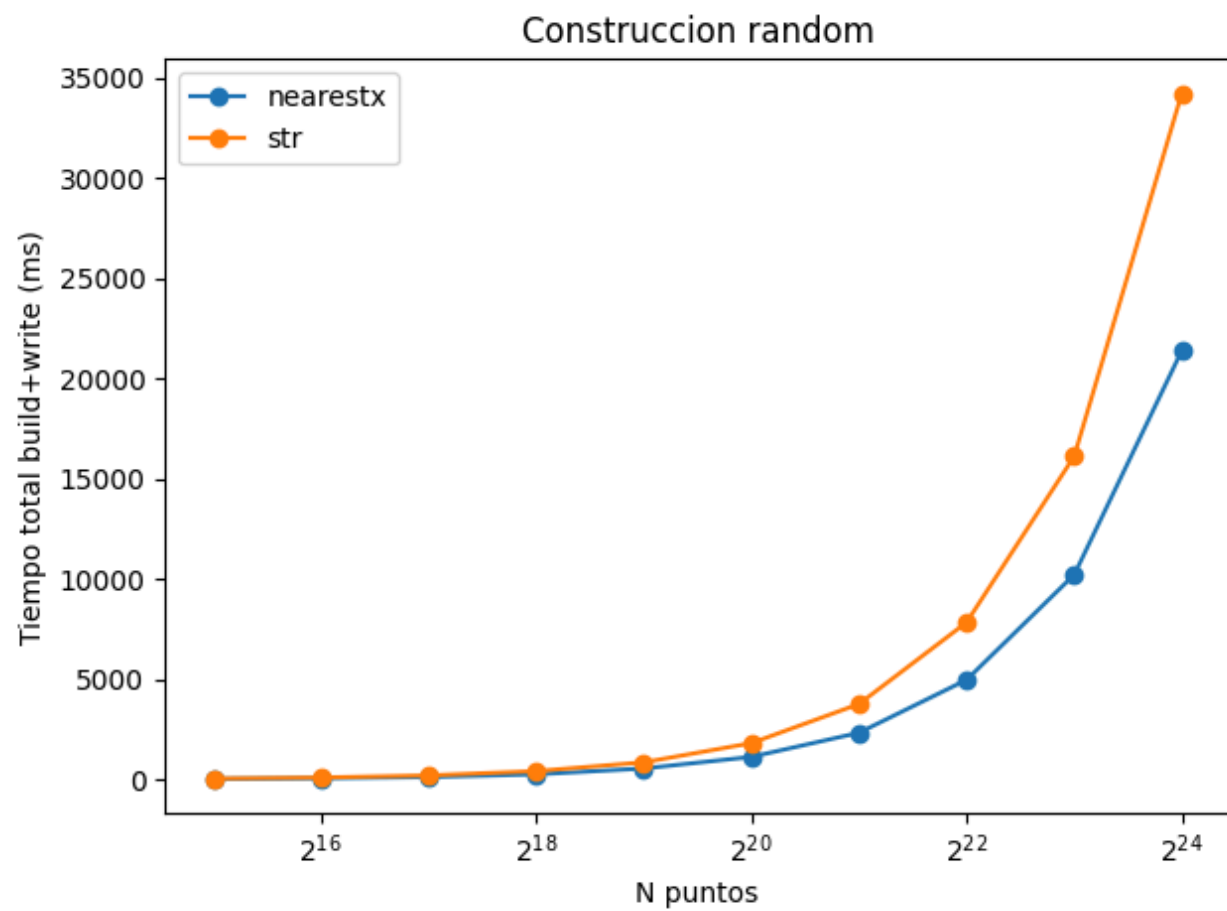


Figura 1: Tiempo de construcción en random.

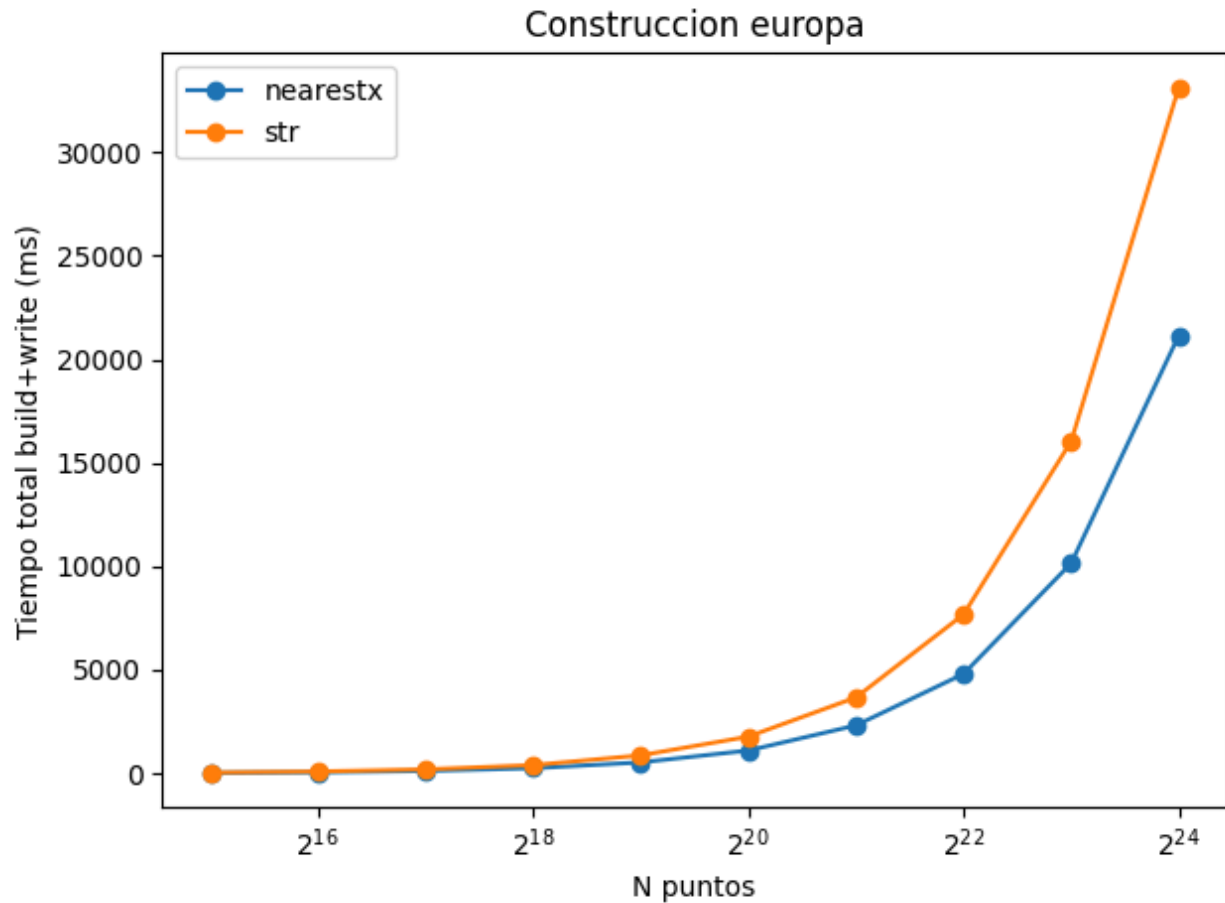


Figura 2: Tiempo de construcción en Europa.

La separación entre ambas curvas se vuelve más visible desde los tamaños grandes, especialmente desde $N = 2^{21}$.

En ambos datasets, el tiempo de construcción aumenta con N . Para $N = 2^{24}$, STR demora más que Nearest-X: en random, 34226.894 ms frente a 21394.891 ms; en Europa, 33126.152 ms frente a 21149.141 ms. Ambos métodos producen la misma cantidad de nodos y la misma altura para el mismo valor de N .

4.2. Lecturas promedio de nodo en consultas

La Tabla 2 muestra el promedio de lecturas de nodo y el tiempo promedio de consulta para los árboles con $N = 2^{24}$.

Dataset	Método	s	I/O promedio	Tiempo promedio (ms)
random	Nearest-X	0.0025	209.630	1.767
random	Nearest-X	0.005	416.120	3.494
random	Nearest-X	0.01	829.820	6.960
random	Nearest-X	0.025	2070.300	18.087
random	Nearest-X	0.05	4136.580	37.413
random	STR	0.0025	12.170	0.154
random	STR	0.005	15.790	0.203
random	STR	0.01	26.760	0.388
random	STR	0.025	84.620	1.584
random	STR	0.05	263.100	5.633
europa	Nearest-X	0.0025	134.610	1.127
europa	Nearest-X	0.005	274.430	2.275
europa	Nearest-X	0.01	557.900	4.722
europa	Nearest-X	0.025	1513.330	14.157
europa	Nearest-X	0.05	3342.340	35.486
europa	STR	0.0025	9.790	0.125
europa	STR	0.005	15.620	0.237
europa	STR	0.01	34.170	0.617
europa	STR	0.025	159.600	3.413
europa	STR	0.05	649.230	14.633

Tabla 2: Resumen de promedios de lectura y consulta.

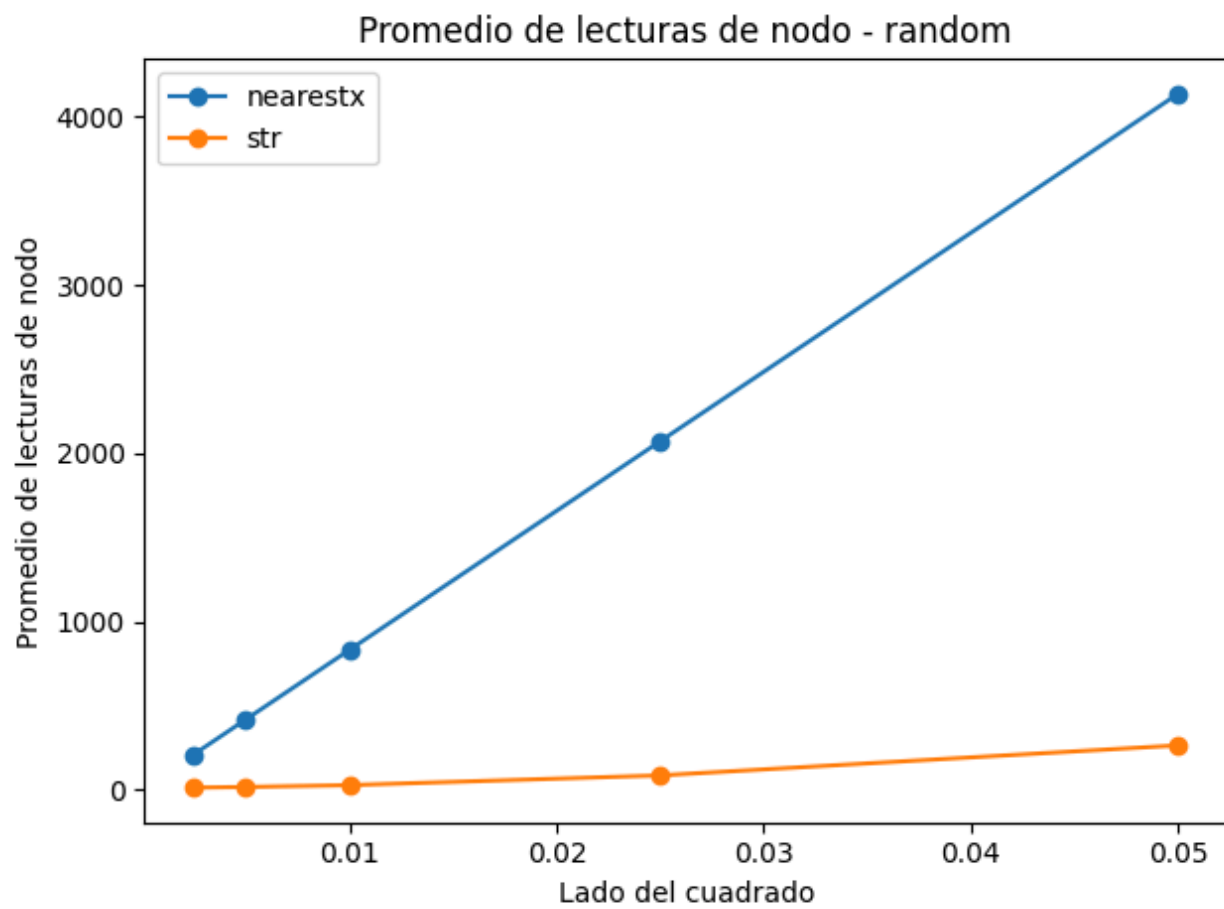


Figura 3: Lecturas promedio de nodo en random.

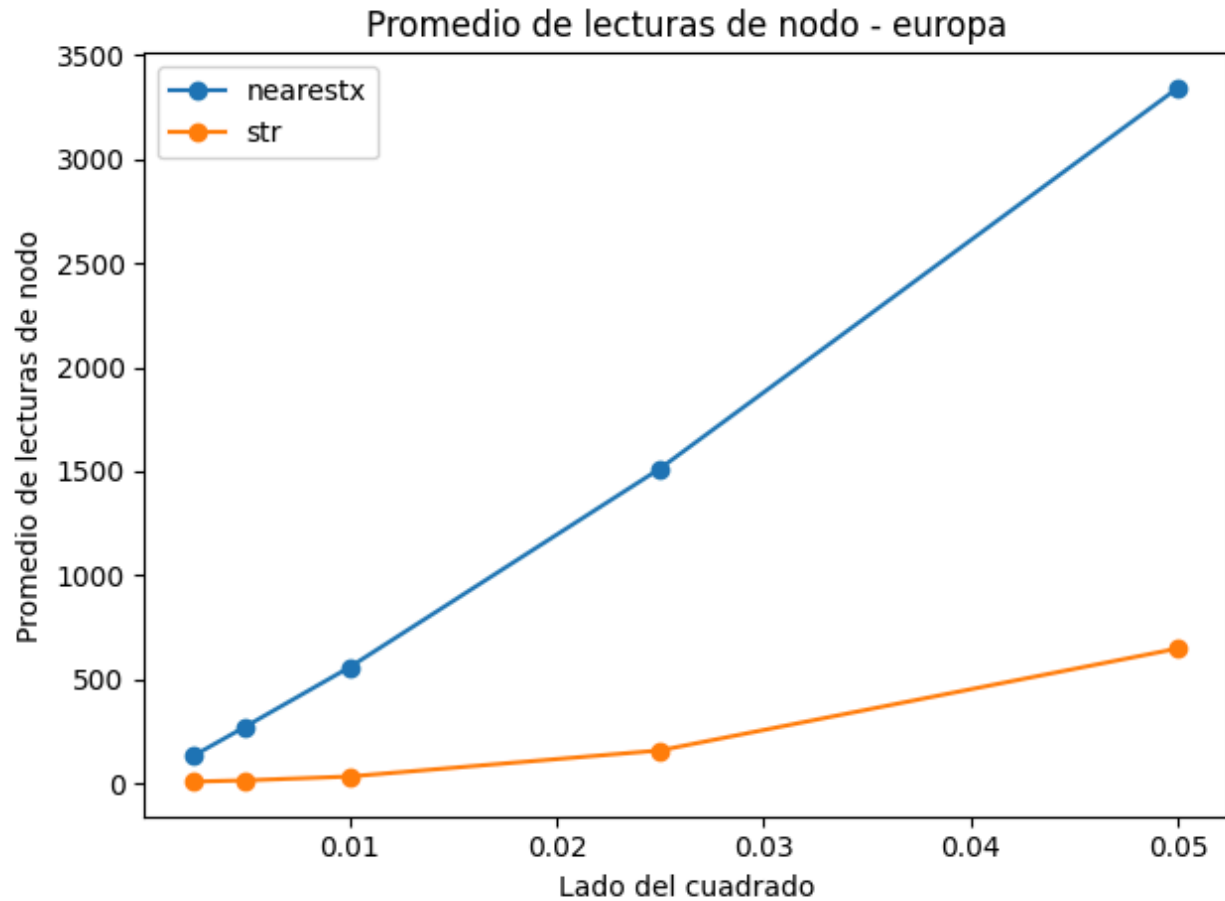


Figura 4: Lecturas promedio de nodo en Europa.

En todos los valores de s , STR realiza menos lecturas promedio de nodo que Nearest-X. La absoluta diferencia aumenta en consultas más grandes. En random, para $s = 0.05$, Nearest-X lee 4136.580 nodos en promedio y STR lee 263.100. En Europa, para el mismo lado, Nearest-X lee 3342.340 nodos y STR lee 649.230.

4.3. Puntos reportados

La Tabla 3 muestra el promedio y la desviación estándar de los puntos reportados por consulta.

Dataset	Método	s	Puntos promedio	Desv. estándar
random	Nearest-X	0.0025	103.570	10.598
random	Nearest-X	0.005	418.220	21.207
random	Nearest-X	0.01	1681.550	39.188
random	Nearest-X	0.025	10515.680	102.250
random	Nearest-X	0.05	41997.130	205.293
random	STR	0.0025	103.570	10.598
random	STR	0.005	418.220	21.207
random	STR	0.01	1681.550	39.188
random	STR	0.025	10515.680	102.250
random	STR	0.05	41997.130	205.293
europa	Nearest-X	0.0025	171.380	744.297
europa	Nearest-X	0.005	981.590	4854.622
europa	Nearest-X	0.01	3920.760	16124.753
europa	Nearest-X	0.025	27182.630	115950.056
europa	Nearest-X	0.05	122114.340	420078.869
europa	STR	0.0025	171.380	744.297
europa	STR	0.005	981.590	4854.622
europa	STR	0.01	3920.760	16124.753
europa	STR	0.025	27182.630	115950.056
europa	STR	0.05	122114.340	420078.869

Tabla 3: Resumen del promedio y la desviación estándar de los puntos reportados por consulta.

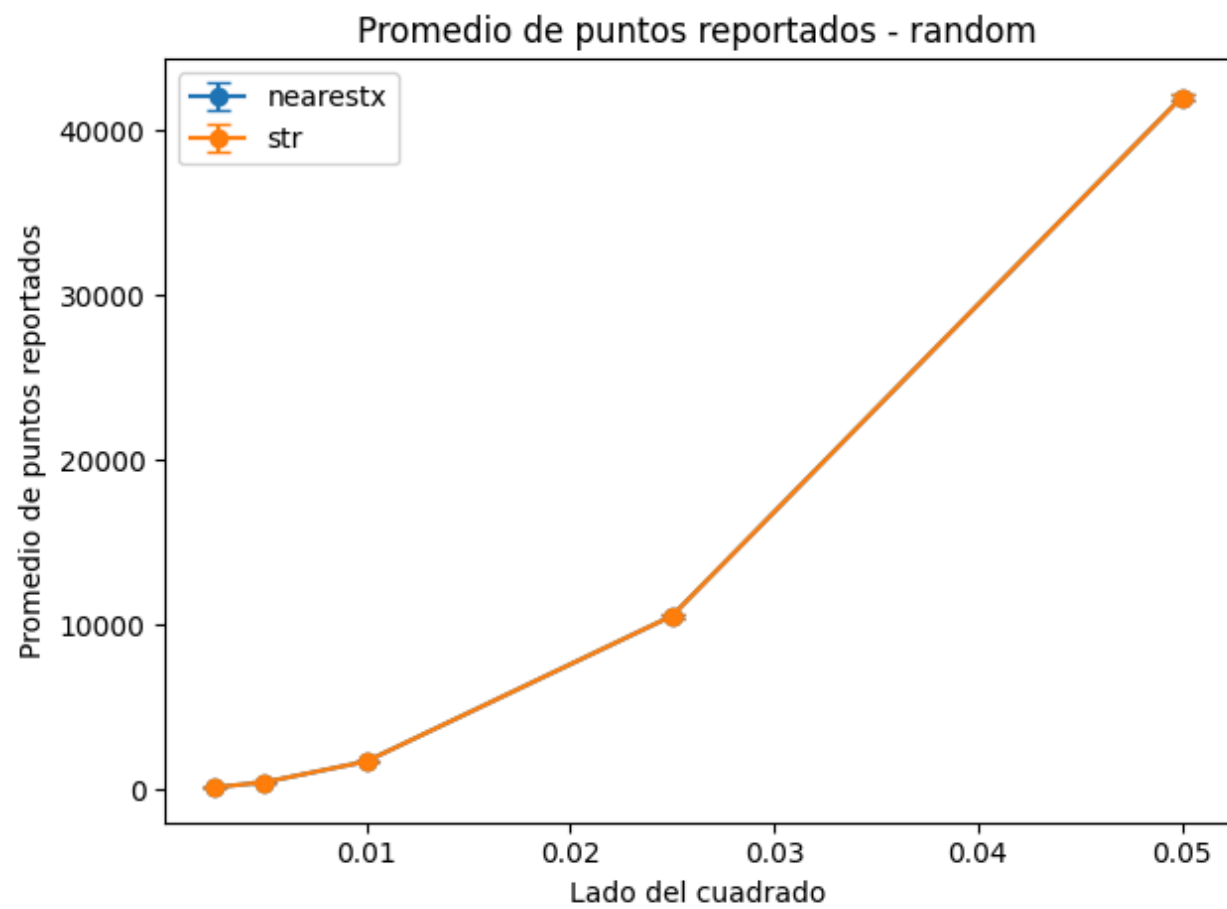


Figura 5: Puntos reportados en random.

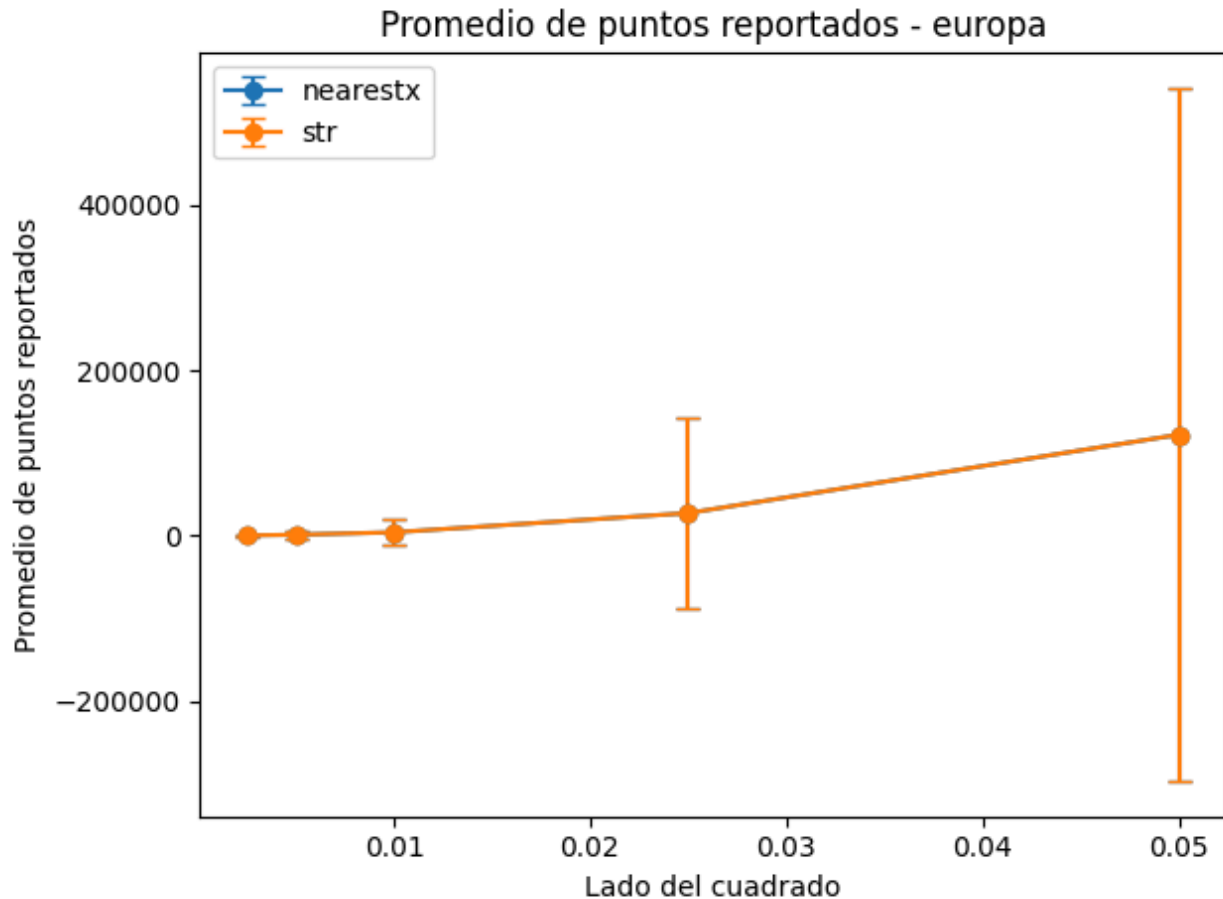


Figura 6: Puntos reportados en Europa.

Nota: las barras de error muestran media $\pm$ desviación estándar. En Europa, algunas barras se extienden bajo cero porque la desviación estándar supera al promedio; esto no representa una cantidad negativa de puntos.

Para cada dataset y cada tamaño de consulta, ambos métodos reportan el mismo promedio de puntos hasta la precisión mostrada. En random, la desviación estándar es baja en relación con el promedio. En Europa, la desviación estándar es mucho mayor, especialmente para lados grandes. En el gráfico de Europa, las barras de error representan media más/menos desviación estándar; si una barra baja de cero, eso no indica puntos negativos, sino una desviación estándar mayor que el promedio.

5. Análisis

Los resultados cumplen la tendencia esperada. Nearest-X construye más rápido porque realiza una agrupación espacial más simple: ordena por la coordenada X y empaqueta entradas consecutivas. STR, en cambio, debe ordenar por X, dividir en franjas y ordenar por Y dentro de cada franja. Por esta razón, su tiempo de construcción es mayor en ambos datasets.

El costo adicional de STR se compensa en las consultas. En un R-tree, una consulta por rango avanza hacia todos los hijos cuyos MBR intersectan el rectángulo consultado. Por lo tanto, mientras más grandes o solapados sean los MBR, mayor es la probabilidad de leer ramas que finalmente contienen pocos o ningún punto útil. Nearest-X puede generar nodos compactos en X pero extendidos en Y, lo

que aumenta estas intersecciones falsas. STR produce una partición más bidimensional, con MBR más localizados, y por eso reduce las lecturas promedio de nodo.

La diferencia entre ambos métodos se ve con claridad en $s = 0.01$. En random, Nearest-X realiza 829.820 lecturas promedio y STR solo 26.760. En Europa, Nearest-X realiza 557.900 lecturas promedio y STR 34.170. Esto muestra que la mejora no depende solo del dataset uniforme, sino también aparece en datos reales normalizados.

La cantidad de puntos reportados sirve como control de corrección. Para cada dataset y valor de s , Nearest-X y STR reportan la misma cantidad promedio de puntos. Esto indica que ambos árboles representan el mismo conjunto de datos y que la diferencia observada está en el costo de acceso, no en el resultado de la consulta.

En el dataset random, los puntos encontrados son coherentes con el área del cuadrado. Por ejemplo, para $s = 0.01$, el área de consulta es 0.0001 ; con $N = 16777216$, se esperan aproximadamente 1677.7 puntos si la distribución es uniforme. El promedio medido fue 1681.550, muy cercano a ese valor. Esto refuerza que la búsqueda está recuperando una cantidad esperable de puntos.

En el dataset Europa, la desviación estándar de puntos reportados es mucho mayor porque los datos no están distribuidos uniformemente. Algunas consultas caen sobre zonas con alta densidad de edificios y otras sobre zonas con pocos puntos. Esta concentración espacial también explica que los resultados no sigan tan directamente el área del cuadrado como ocurre en random.

Desde el punto de vista del modelo de memoria externa, el resultado más relevante es la reducción de lecturas de bloque. Como cada nodo ocupa un bloque lógico, leer menos nodos equivale directamente a reducir I/O. En ese sentido, STR produce una estructura más cara de construir, pero más adecuada para consultas por rango sobre datos espaciales.

Esto coincide con el modelo de memoria externa visto en el curso: cuando cada bloque contiene un nodo, el costo dominante de una consulta no es recorrer arreglos en CPU, sino reducir la cantidad de bloques leídos.

6. Conclusión

Se implementó un R-tree estático para puntos 2D con nodos de 4096 bytes, serialización binaria y consultas desde disco. También se implementaron los dos métodos de *bulk-loading* pedidos: Nearest-X y STR. La construcción se realizó primero en memoria, usando un vector de nodos, y luego se escribió el árbol completo a un archivo binario para ejecutar las consultas por rango.

La hipótesis se cumple. STR tiene mayor costo de construcción más escritura, pero genera árboles más eficientes para consultas por rango. Nearest-X es más rápido al construir, aunque sus árboles requieren muchas más lecturas de nodo durante consultas. La razón principal es la calidad del agrupamiento espacial: STR considera X e Y al empaquetar nodos, mientras que Nearest-X usa una ordenación unidimensional.

Los resultados también muestran que ambos métodos retornan los mismos puntos para las mismas consultas. Por lo tanto, la comparación experimental no indica diferencias de corrección, sino diferencias de eficiencia en acceso a disco.

Como trabajo futuro, se podría mejorar la serialización escribiendo campo a campo en vez de escribir structs crudos, lo que haría el formato más portable entre arquitecturas. También se podría agregar un *buffer manager* para estudiar el efecto de caché de bloques, ejecutar múltiples semillas para medir variabilidad entre corridas, e incluir visualizaciones de los MBR generados por cada método. En esta versión no se implementó la bonificación con `europa_bonus.bin`.

7. Reproducibilidad

Los resultados usados en este informe corresponden a los archivos presentes en `results/`. Para compilar y ejecutar los experimentos se usaron los siguientes comandos.

```
cmake -S . -B build
cmake --build build
ctest --test-dir build --output-on-failure

.\scripts\run_required_experiments.ps1 -Fresh `
  -Cli .\build\rtree_cli.exe `
  -RandomDataset data\random.bin `
  -EuropaDataset data\europa.bin `
  -ResultsDir results `
  -TreesDir trees `
  -Log results\required_experiments_progress.log

uv run --python 3.12 --with matplotlib python scripts\plot_build_times.py `
  --csv results\build_results.csv `
  --out-dir results

uv run --python 3.12 --with matplotlib python scripts\plot_query_summary.py `
  --csv results\query_summary.csv `
  --out-dir results
```